



BITS Pilani

Pilani Campus

Deep Learning

Deep Neural Network

innovate

achieve

lead

Disclaimer and Acknowledgement



- The content for these slides has been obtained from books and various other source on the Internet
- I here by acknowledge all the contributors for their material and inputs.
- I have provided source information wherever necessary
- I have added and modified the content to suit the requirements of the course

Session Agenda



Popular CNN architectures

Popular CNN architectures



- LeNet (LeCun *et al.*, 1995): achieved good results on early small datasets, the performance and feasibility of training CNNs on larger, more realistic datasets were not established
- AlexNet (Krizhevsky *et al.*, 2012): the first large-scale network deployed to beat conventional computer vision methods on a large-scale vision challenge
- VGG network (Simonyan and Zisserman, 2014): uses number of repeating blocks of elements
- Network in network (NiN) (Lin *et al.*, 2013): convolves whole neural networks patch-wise over inputs
- GoogLeNet (Szegedy *et al.*, 2015): uses networks with multi-branch convolutions
- Residual network (ResNet) (He *et al.*, 2016): one of the most popular off-the-shelf architectures in computer vision
- ResNet blocks (Xie *et al.*, 2017) : sparser connections
- DenseNet (Huang *et al.*, 2017): a generalization of the residual architecture

Popular CNN architectures

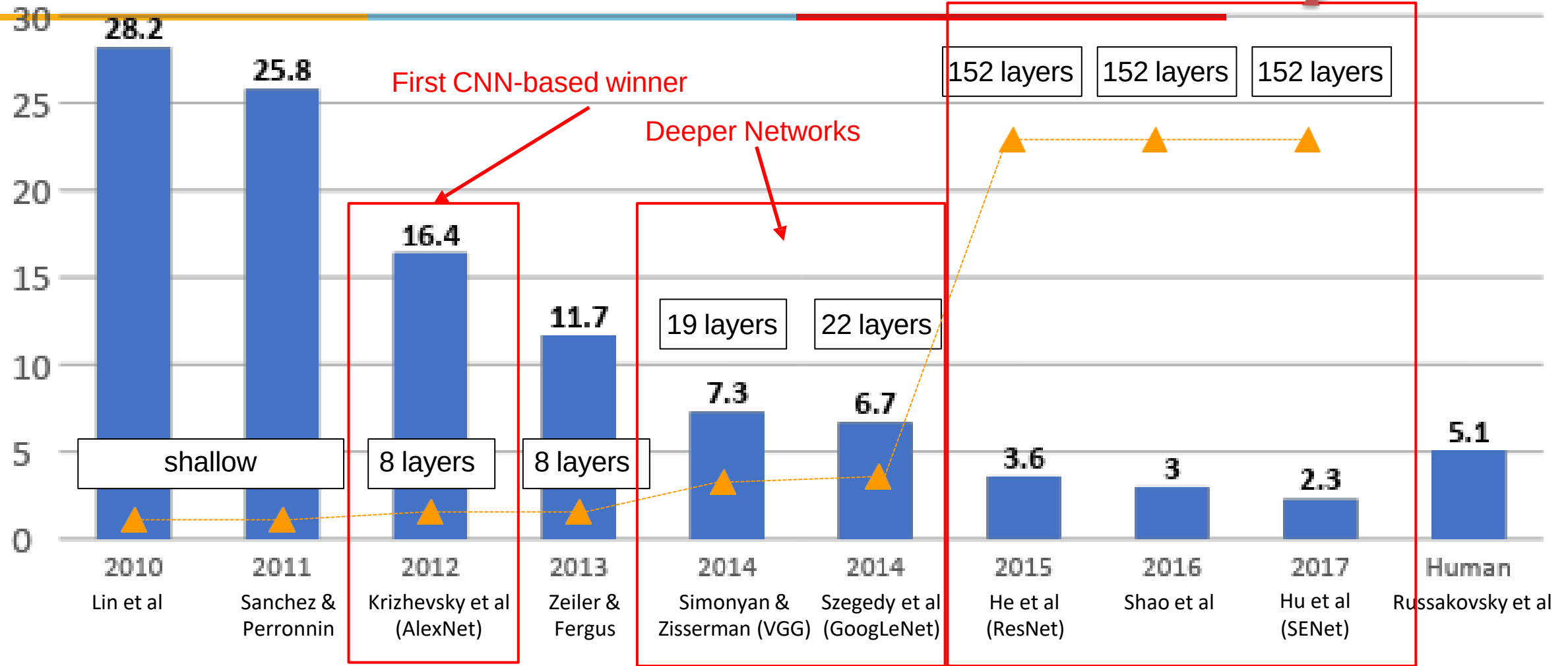


- WordNet is a lexical database of semantic relations between words and used for language processing by the researchers
- ImageNet developed by Princeton University researchers, which is a large image data base
- ImageNet was conducting ImageNet Large Scale Visual Recognition Challenge (ILSVRC) since 2010.
 - A large set of images (~1 million) are offered as training and validation data to perform object localization and classification (~1000 classes)
 - Results are reported in terms of top-5 error rate. E.g. an image of car is used as the test image and the predicted output of the model does not have car in the top-5 classes that it predicts, then it is considered as an error
- The popular winner architectures are AlexNet (2012), ZFNet (2013), VGG (2014-first runner up), GoogLeNet (2014) and ResNet (2015)
- AlexNet, ZFNet and VGG are similar to LeNet-5 but much larger and much deeper and with few other changes. They take 224x224 RGB images as input.

ImageNet Large Scale Visual Recognition Challenge (ILSVRC) winners



Revolution in Depth



LeNet-5 (First popular CNN architecture -1998)



PROC. OF THE IEEE, NOVEMBER 1998

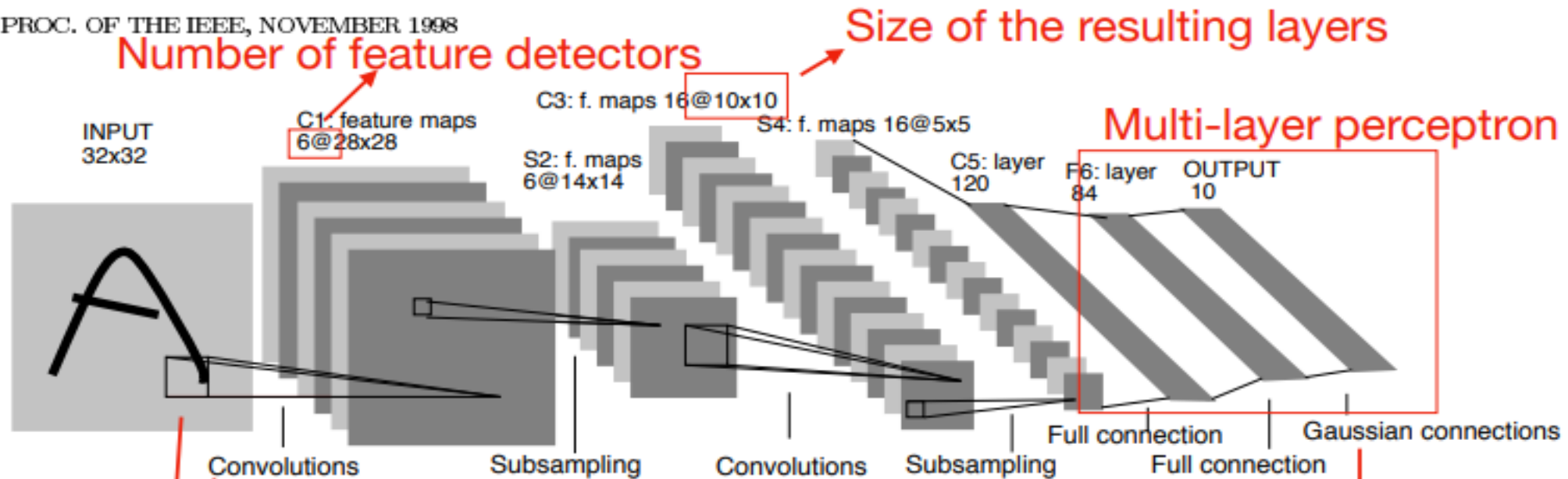


Fig. 2. Architecture of LeNet-5, a Convolutional Neural Network, here for digits recognition. Each plane is a feature map, i.e. a set of units whose weights are constrained to be identical.

nowadays called "pooling"

"Feature detectors" (weight matrices)
that are being reused ("weight sharing")
=> also called "kernel" or "filter"

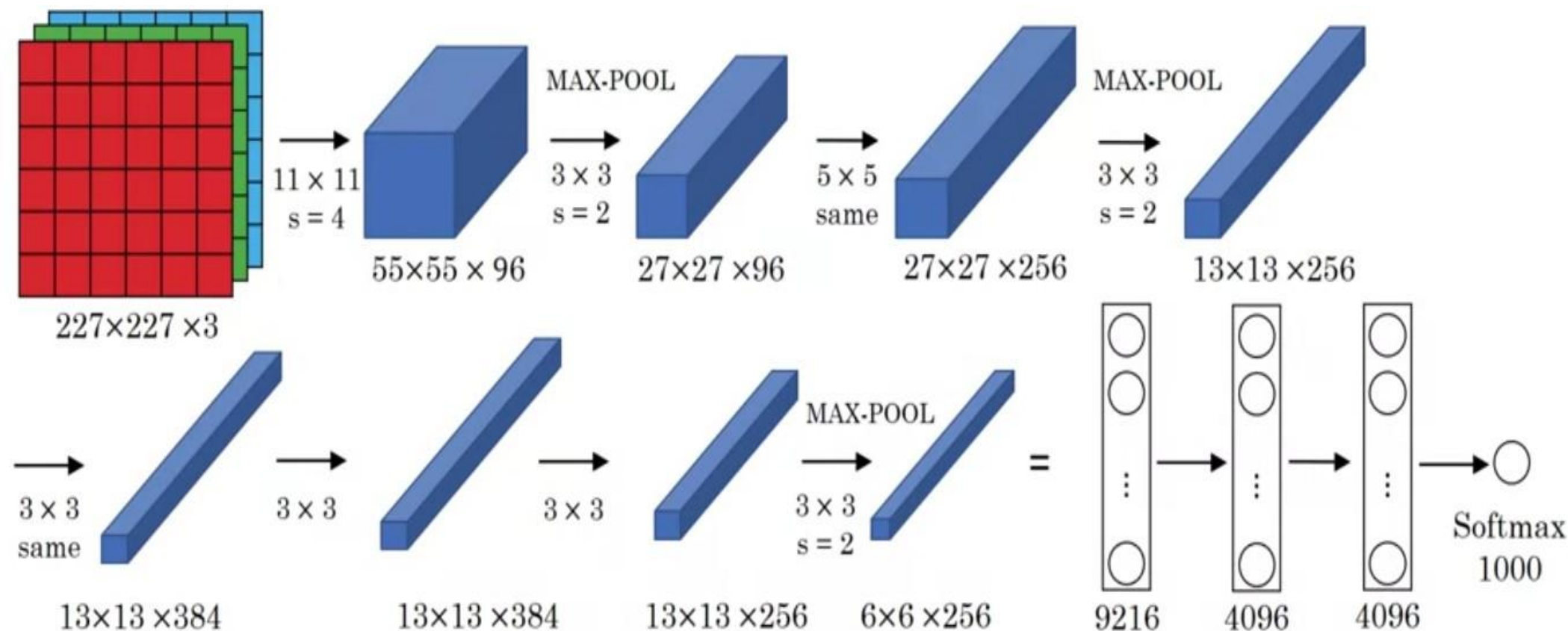
basically a fully-connected
layer + MSE loss
(nowadays common to use
fc-layer + softmax
+ cross entropy)

AlexNet (Krizhevsky et al 2012)



- AlexNet competed in the ImageNet Large Scale Visual Recognition Challenge on September 30, 2012. The network achieved a top-5 error of 15.3%, more than 10.8 percentage points lower than that of the runner up
- Details/Retrospectives:
 - First use of ReLU
 - Used Local Response Normalisation (LRN) layers (not common anymore)
 - Approx. 60 million parameters are learned
 - heavy data augmentation : e.g flipping, clipping, and color changes
 - dropout 0.5
 - batch size 128
 - SGD Momentum 0.9
 - Learning rate $1e-2$, reduced by 10 manually when val accuracy plateaus
 - L2 weight decay $5e-4$
 - 7 CNN ensemble: 18.2% → 15.4%

AlexNet



Architecture:

CONV1

MAX POOL1

NORM1

CONV2

MAX POOL2

NORM2

CONV3

CONV4

CONV5

Max POOL3

FC6

FC7

FC8

[Krizhevsky et al., 2012. ImageNet classification with deep convolutional neural networks]

Andrew Ng

AlexNet



Full (simplified) AlexNet architecture:

[227x227x3] INPUT
[55x55x96] **CONV1**: 96 11x11 filters at stride 4, pad 0
[27x27x96] **MAX POOL1**: 3x3 filters at stride 2
[27x27x96] **NORM1**: Normalization layer
[27x27x256] **CONV2**: 256 5x5 filters at stride 1, pad 2
[13x13x256] **MAX POOL2**: 3x3 filters at stride 2
[13x13x256] **NORM2**: Normalization layer
[13x13x384] **CONV3**: 384 3x3 filters at stride 1, pad 1
[13x13x384] **CONV4**: 384 3x3 filters at stride 1, pad 1
[13x13x256] **CONV5**: 256 3x3 filters at stride 1, pad 1
[6x6x256] **MAX POOL3**: 3x3 filters at stride 2
[4096] **FC6**: 4096 neurons
[4096] **FC7**: 4096 neurons
[1000] **FC8**: 1000 neurons (class scores)

AlexNet



Input: 227x227x3 images

First layer (CONV1): 96 11x11 filters applied at stride 4 =>
output volume size? : $(227-11)/4+1 = 55$

Output volume **[55x55x96]**

Q: What is the total number of parameters in this layer?

Parameters: $(11*11*3 + 1)*96 = 35K$

After CONV1: 55x55x96

Second layer (POOL1): 3x3 filters applied at stride 2

Q: what is the output volume size? : $(55-3)/2+1 = 27$

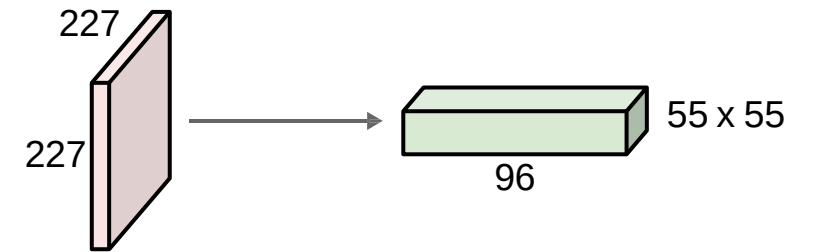
Output volume: 27x27x96

Q: what is the number of parameters in this layer?

Parameters: 0!

After POOL1: 27x27x96

$$O = \left\lfloor \frac{I-K+2p}{s} + 1 \right\rfloor$$

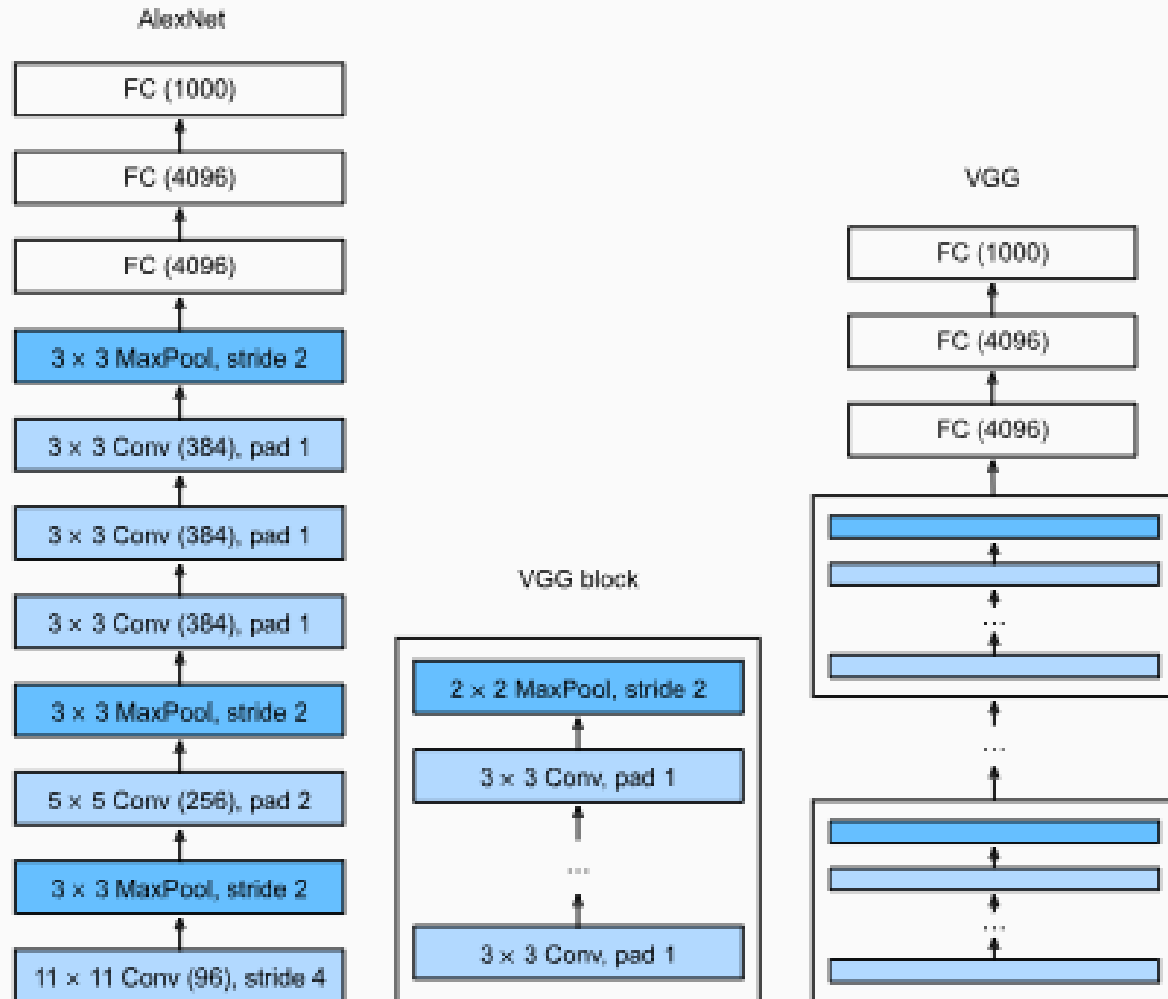


VGGNet (Simonyan and Zisserman, 2014)



- VGG stands for Visual Geometry Group, 16 - 19 layers (VGG16Net)
- **Introduced key properties such as blocks of multiple convolutions and a preference for deep and narrow networks**
- '16' refers to the network's 16 layers that have weights
- Details
 - ILSVRC'14 2nd in classification, 1st in localization
 - Similar training procedure as AlexNet
 - No Local Response Normalisation (LRN)
 - Use VGG16 or VGG19 (VGG19 only slightly better)
 - Approx. 138 million parameters are learned
 - **All convolutions with a 3×3 kernel**, stride 1, pad 1
 - All max-pooling layers with a 2×2 kernel, stride 2

VGGNet



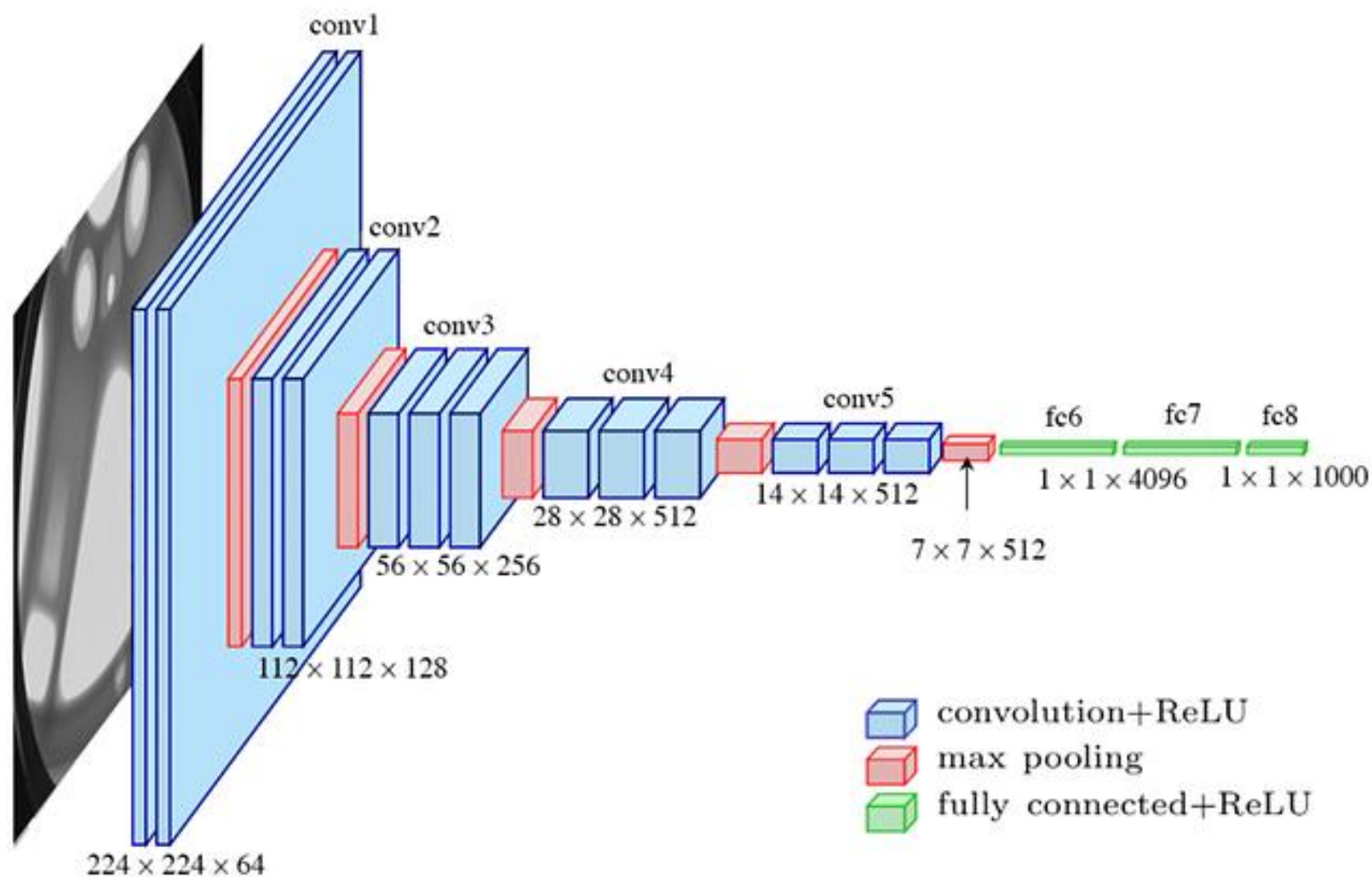
VGG block consists of a *sequence* of convolutions with 3×3 kernels with padding of 1 (keeping height and width) followed by a 2×2 max-pooling layer with stride of 2 (halving height and width after each block).

VGG16

innovate

achieve

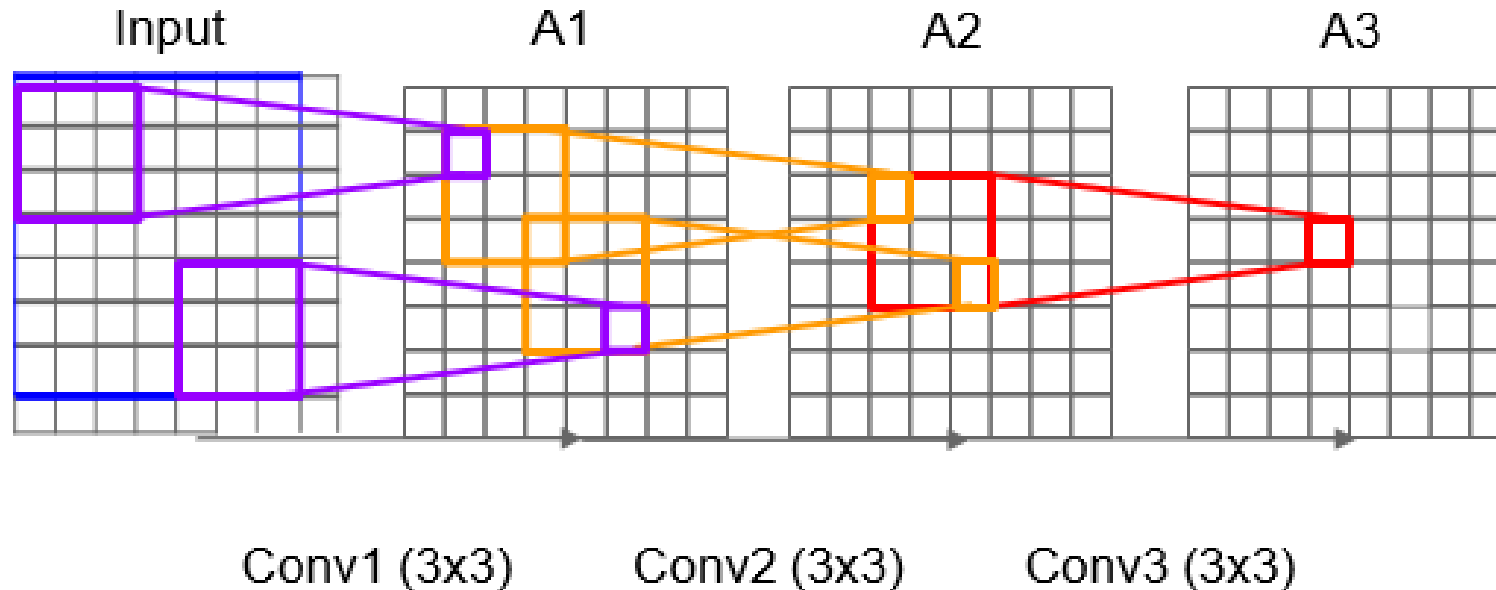
lead



Why use smaller filters? (3x3 conv)

Stack of three 3x3 conv (stride 1) layers has same **effective receptive field** as one 7x7 conv layer

Q: What is the effective receptive field of three 3x3 conv (stride 1) layers?



1 × 1 convolution



6 × 6

1	2	3	6	5	8
3	5	5	1	3	4
2	1	3	4	9	3
4	7	8	5	7	9
1	5	3	7	4	8
5	4	9	8	3	5

6 × 6 × 32

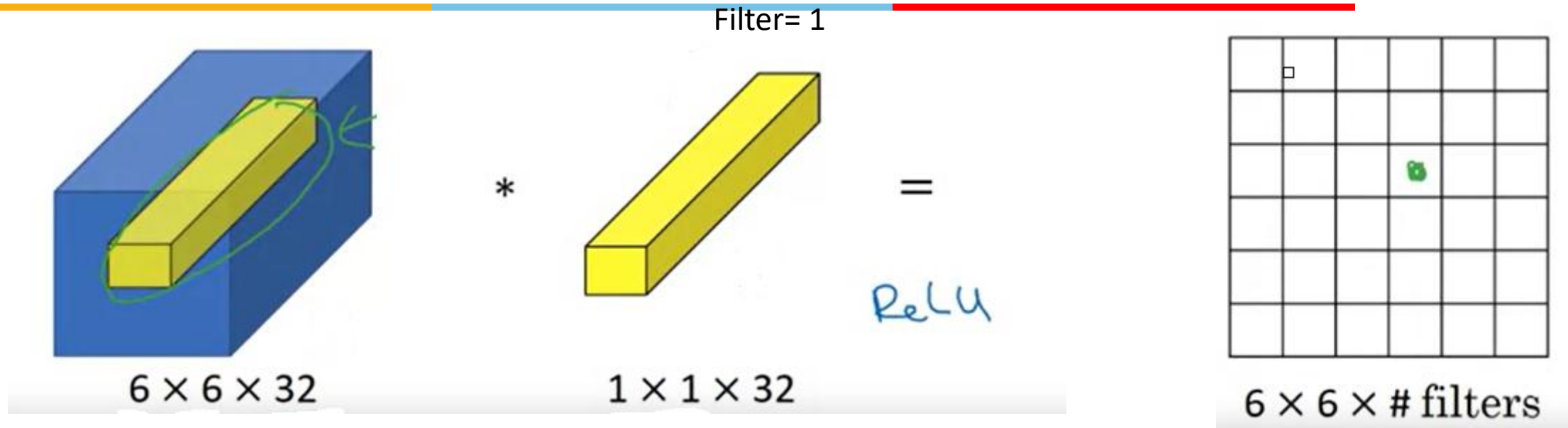
*

2

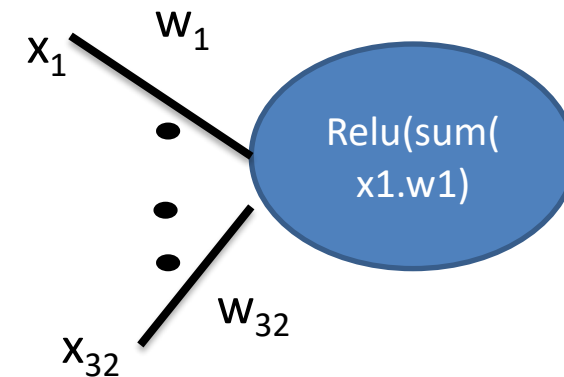
=

2	4	6		

1×1 convolution

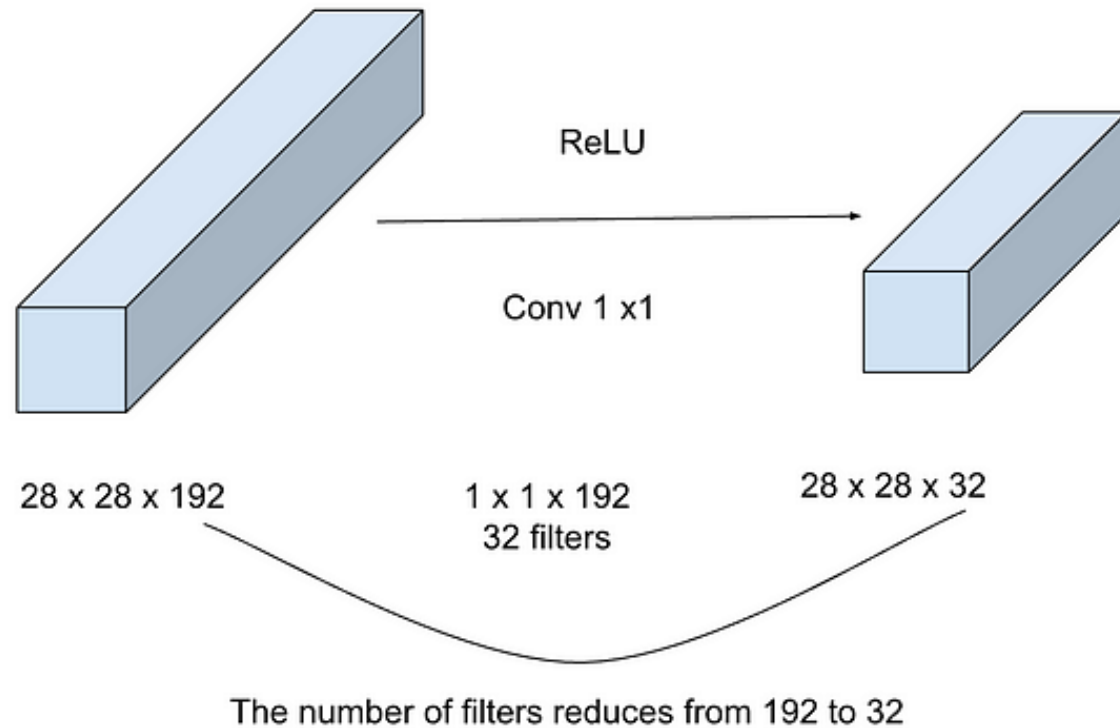


$(1 \times 1 \times 32) \times 6 \times 6 \rightarrow$ Elementwise multiplication for 32 channels and addition to get single pixel value for the output and this operation repeated $6 \times 6 = 36$ times



1×1 convolution

Used to reduce the depth(no of channels)



Network in Network

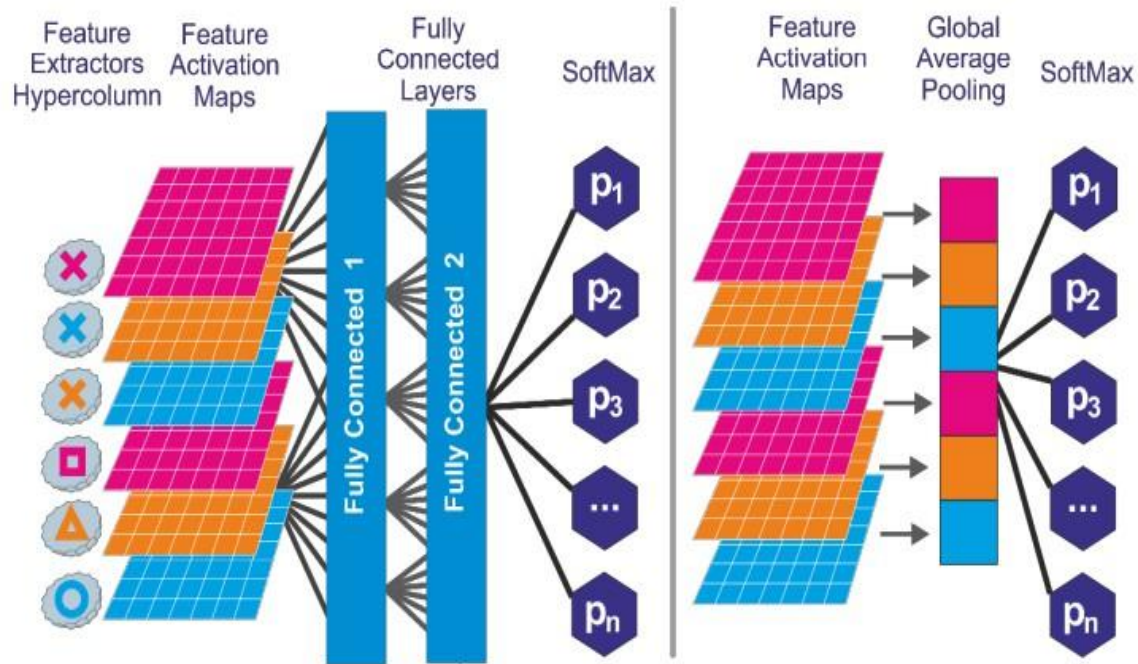


Figure 16: Global average pooling layer replacing the fully connected layers. The output layer implements a Softmax operation with $p_1, p_2, \dots, p_n$ the predicted probabilities for each class.

- One feature map for each corresponding category of the classification task in the last mlpconv layer
- Average of each feature map, and the resulting vector is fed directly into the softmax layer
- Enforces correspondences between feature maps and categories.
- No parameter to optimize in the global average pooling thus overfitting is avoided at this layer unlike FC layer

Network in Network

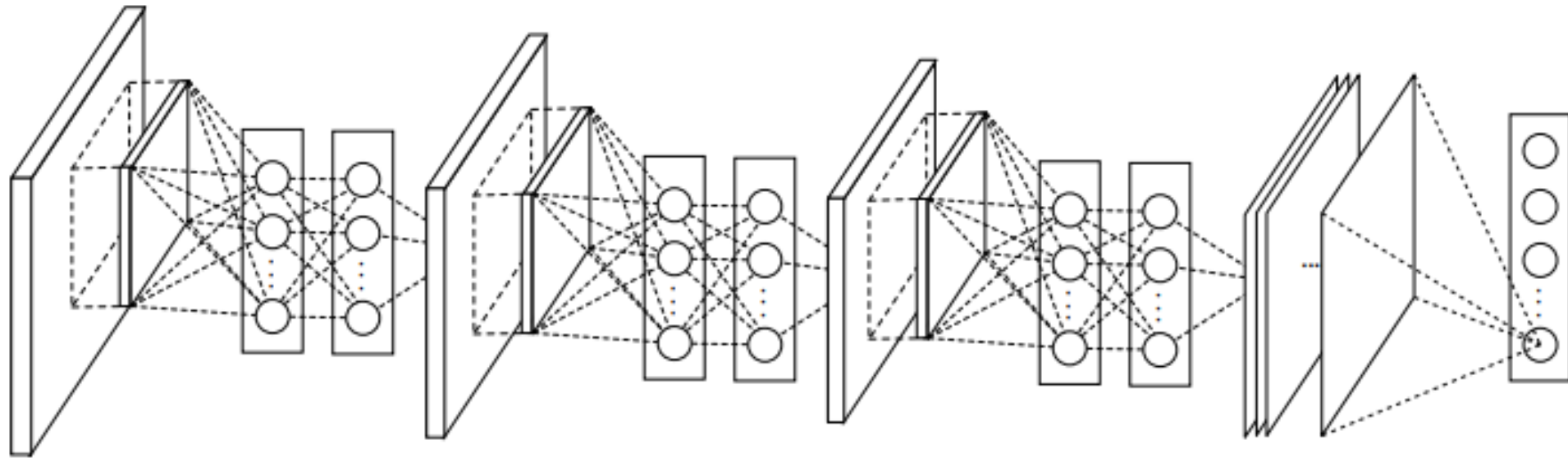
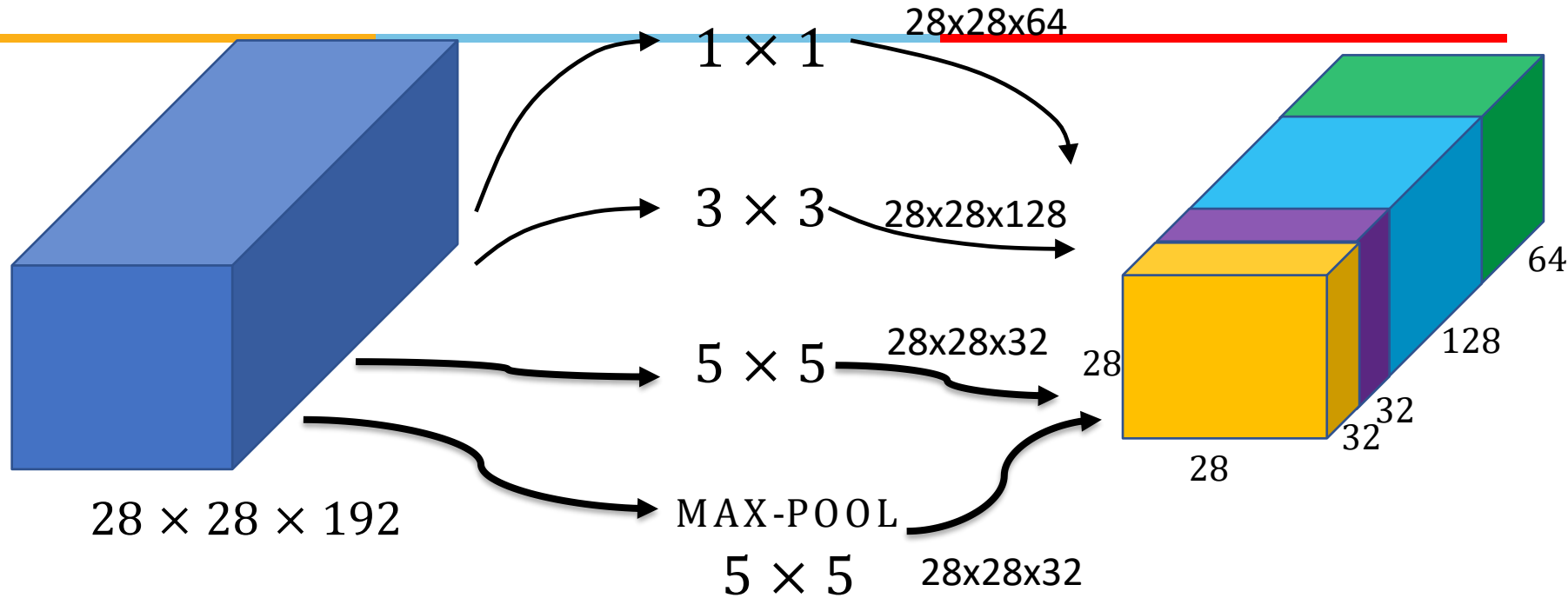


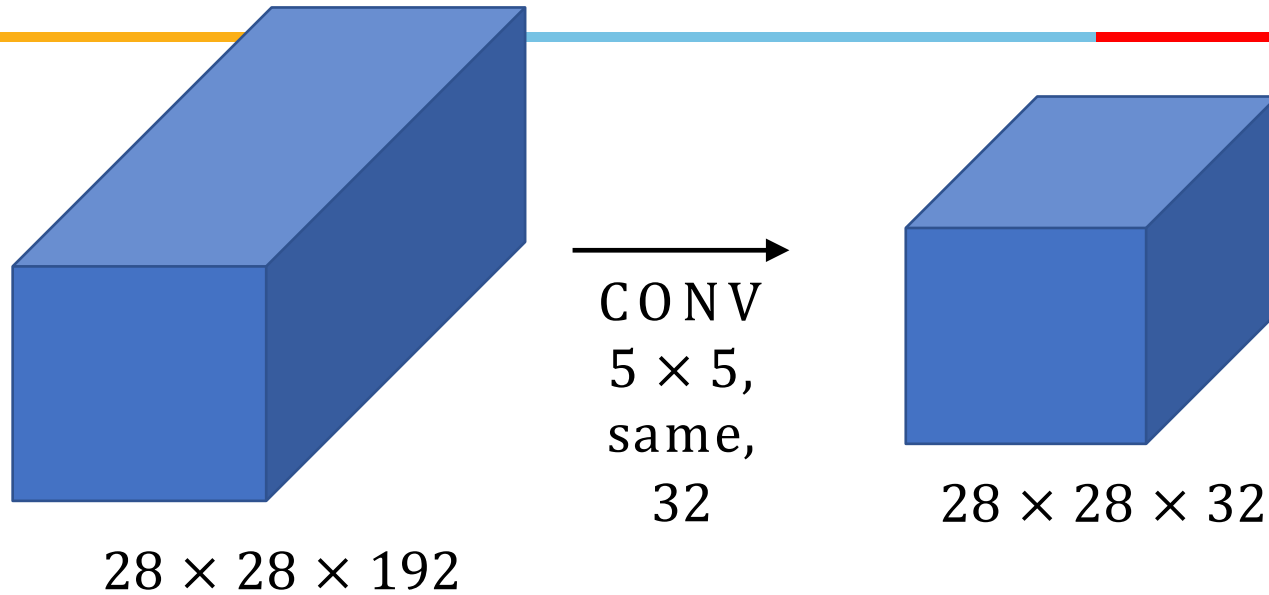
Figure 2: The overall structure of Network In Network. In this paper the NINs include the stacking of three mlpconv layers and one global average pooling layer.

Motivation for inception network



- Apply parallel filter operations on the input from previous layer.
- Multiple receptive field sizes for convolution (1×1 , 3×3 , 5×5) and Pooling operation (3×3)
- Concatenate all filter outputs together channel-wise.
- Very expensive compute
- 1×1 convolutions to reduce feature channel size

Motivation for inception network

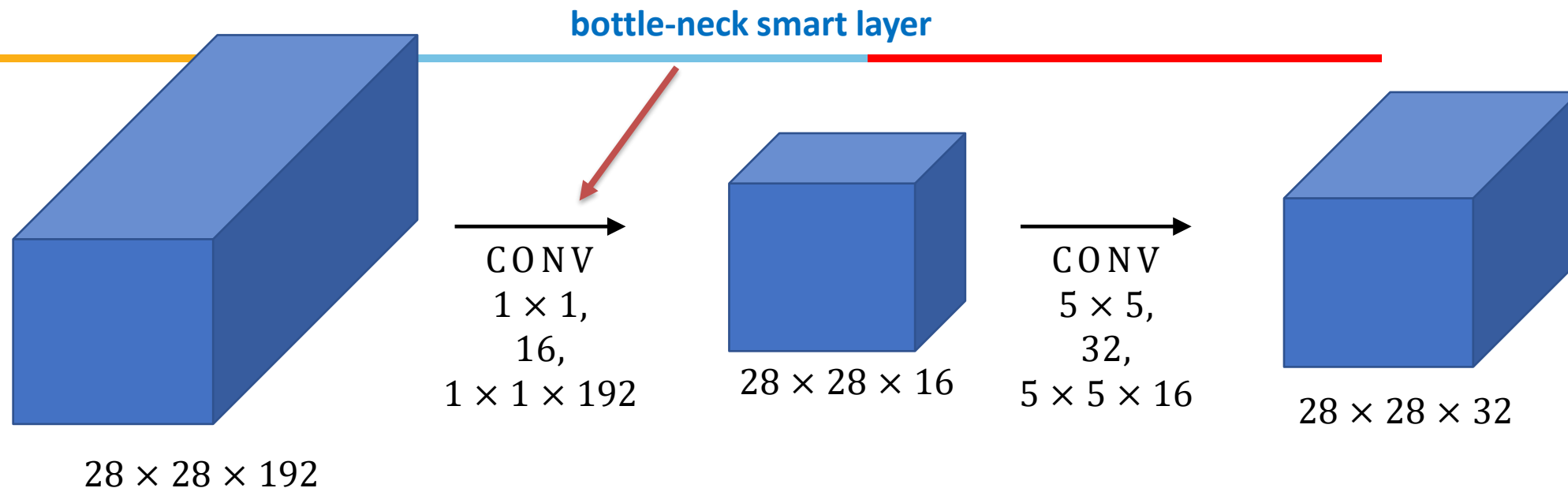


No of multiplication = Output size * filter size

$(28 \times 28 \times 32) \times (5 \times 5 \times 192) \approx \mathbf{120 \text{ million calculations}}$

Andrew Ng

Using 1×1 convolution



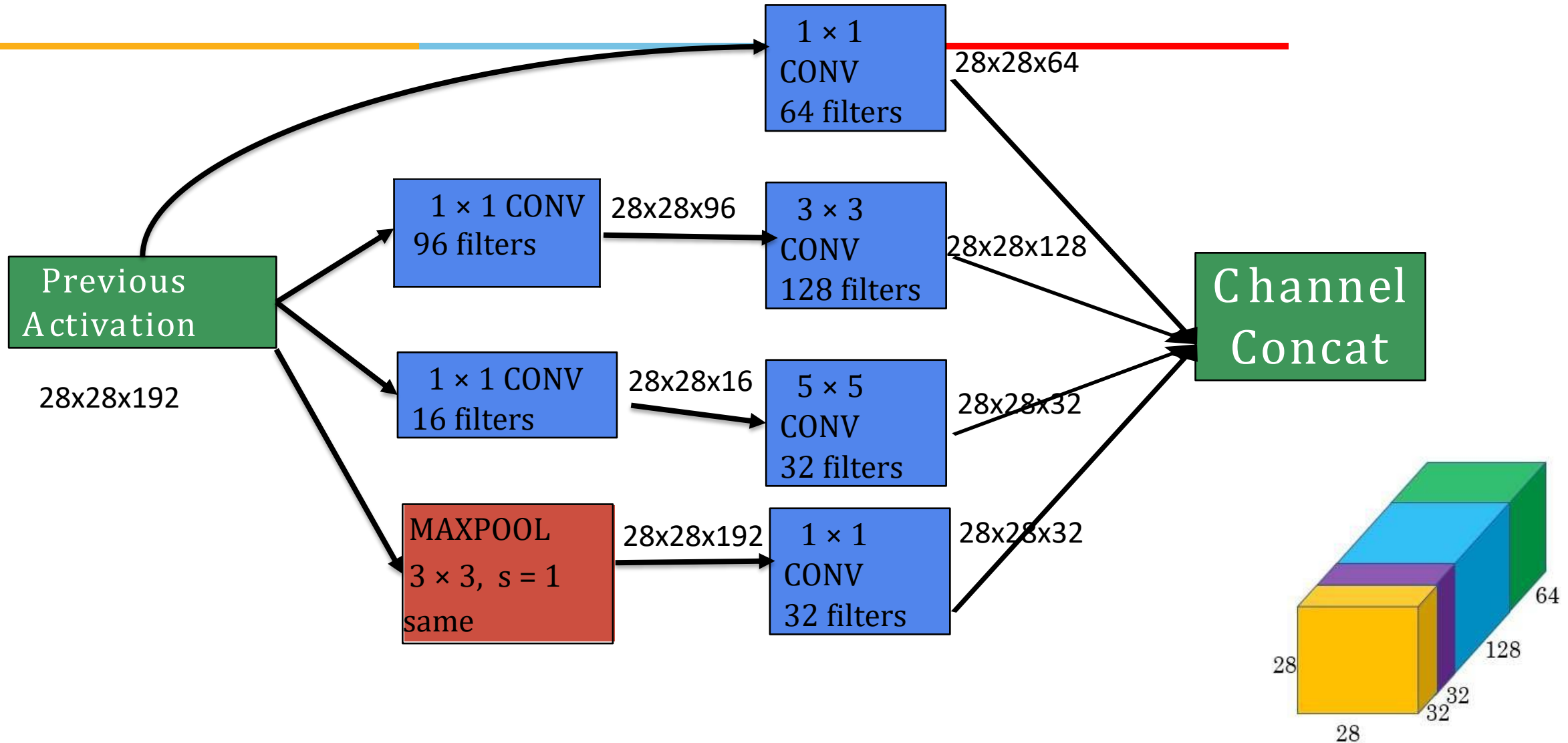
$(1 \times 1 \times 192) \times (28 \times 28) \times 16 \approx \mathbf{2.41 \text{ million}}$ calculations

$(5 \times 5 \times 16) \times (28 \times 28) \times 32 \approx \mathbf{10.04 \text{ million}}$ calculations

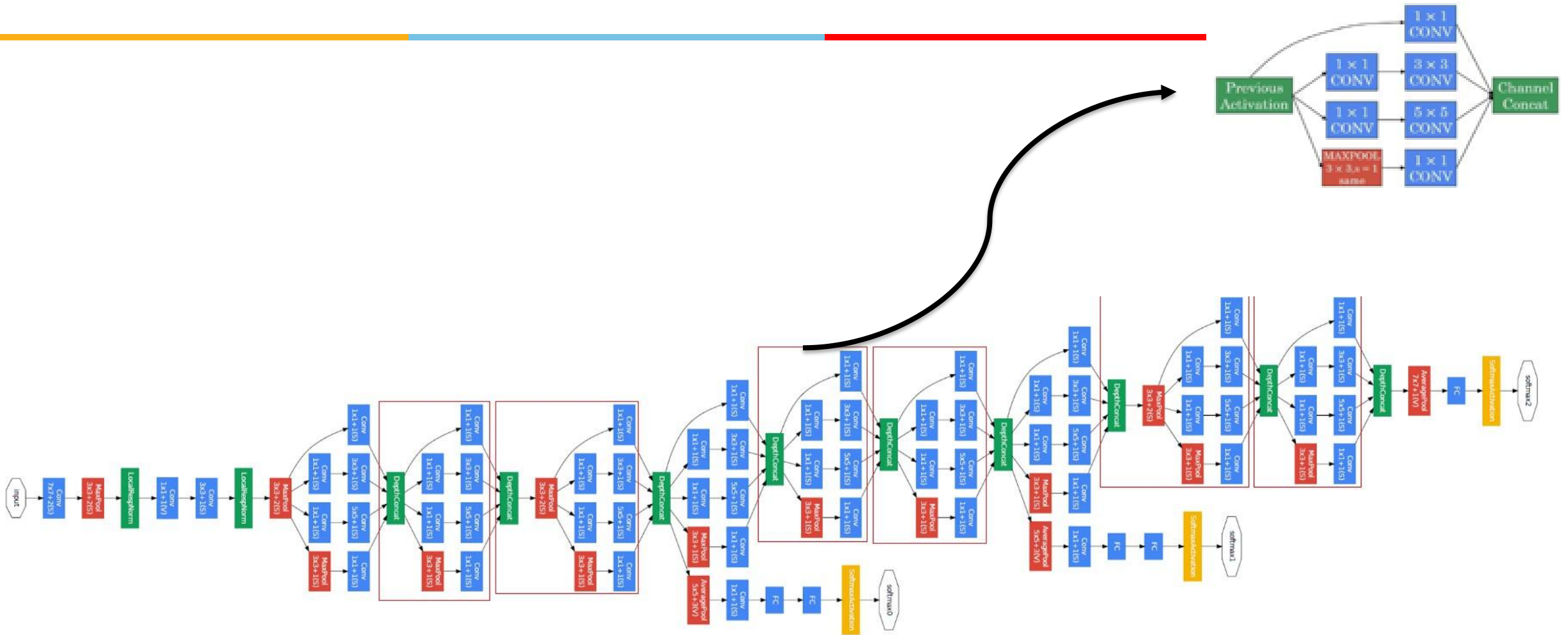
Total 12.45 million calculations

1x1 convolution helps to reduce the depth maintaining the same height and width maintaining the essence of the features. It helps in reducing the computations involved in the convolution layer

Inception module



innovate **achieve** **lead**



Inception Net (Google Net)



- ILSVRC'14 classification winner (6.7% top 5 error)
- 22 layers with weights (including nine inception modules each having three layers each)
- Only 5 million parameters (12x less than AlexNet and 27x less than VGG-16)
- Linear layers at the end
- Max pooling in between, multiple Conv layers between pooling
- Deeper networks, with computational efficiency
- After the last convolutional layer, a global average pooling layer is used that spatially averages across each feature map, before final FC layer

ResNet

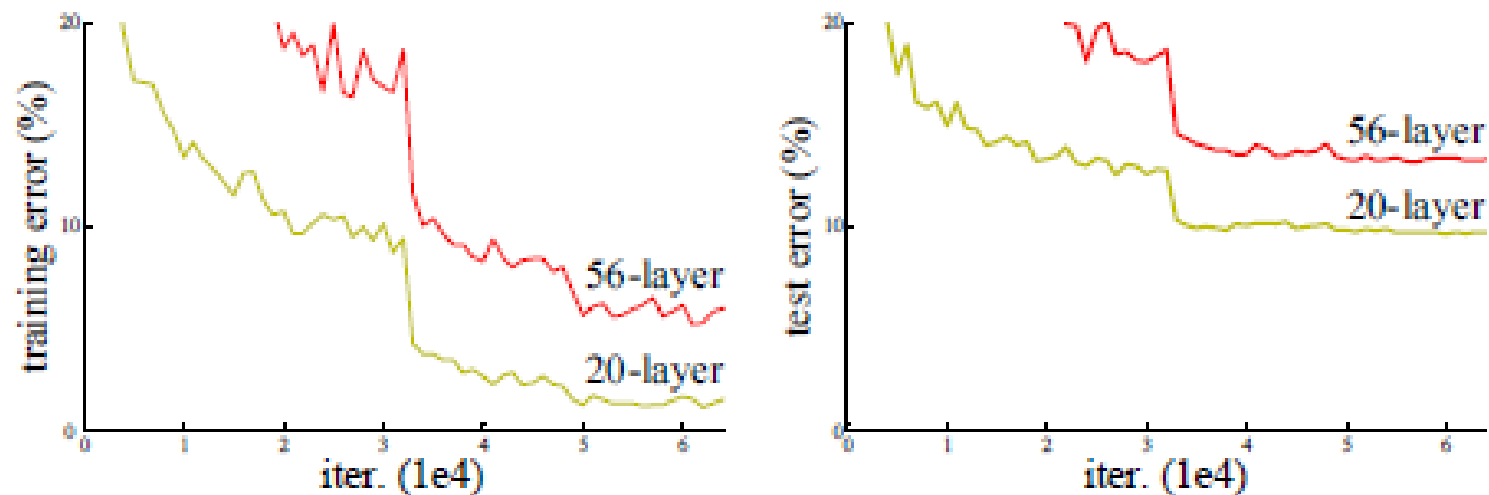


Figure 1. Training error (left) and test error (right) on CIFAR-10 with 20-layer and 56-layer “plain” networks. The deeper network has higher training error, and thus test error. Similar phenomena on ImageNet is presented in Fig. 4.

What happens when we continue stacking deeper layers on a “plain” convolutional neural network?

Fact: Deep models have more representation power (more parameters) than shallower models.

Hypothesis: the problem is an *optimization* problem, **deeper models are harder to optimize**

Skip/Shortcut/Residual connections



- Deeper network → more parameters → needs more training data → each epoch takes more time → training becomes slower
- Many multiplications of random weight matrices with activations in forward propagation so later layer inputs are far from actual input
- Thus, later layers aren't very meaningful because their inputs have been scrambled
- Many multiplications of random weight matrices with gradients in back propagation
- Thus, early layers aren't very meaningful because their gradients have been scrambled
- Gradient descent fluctuates around randomly
- Solution : skip connections
 - Input data to arrive at the later layers of the network and make their inputs more meaningful
 - Loss gradients to arrive at the early layers and make their updates more meaningful

ResNet

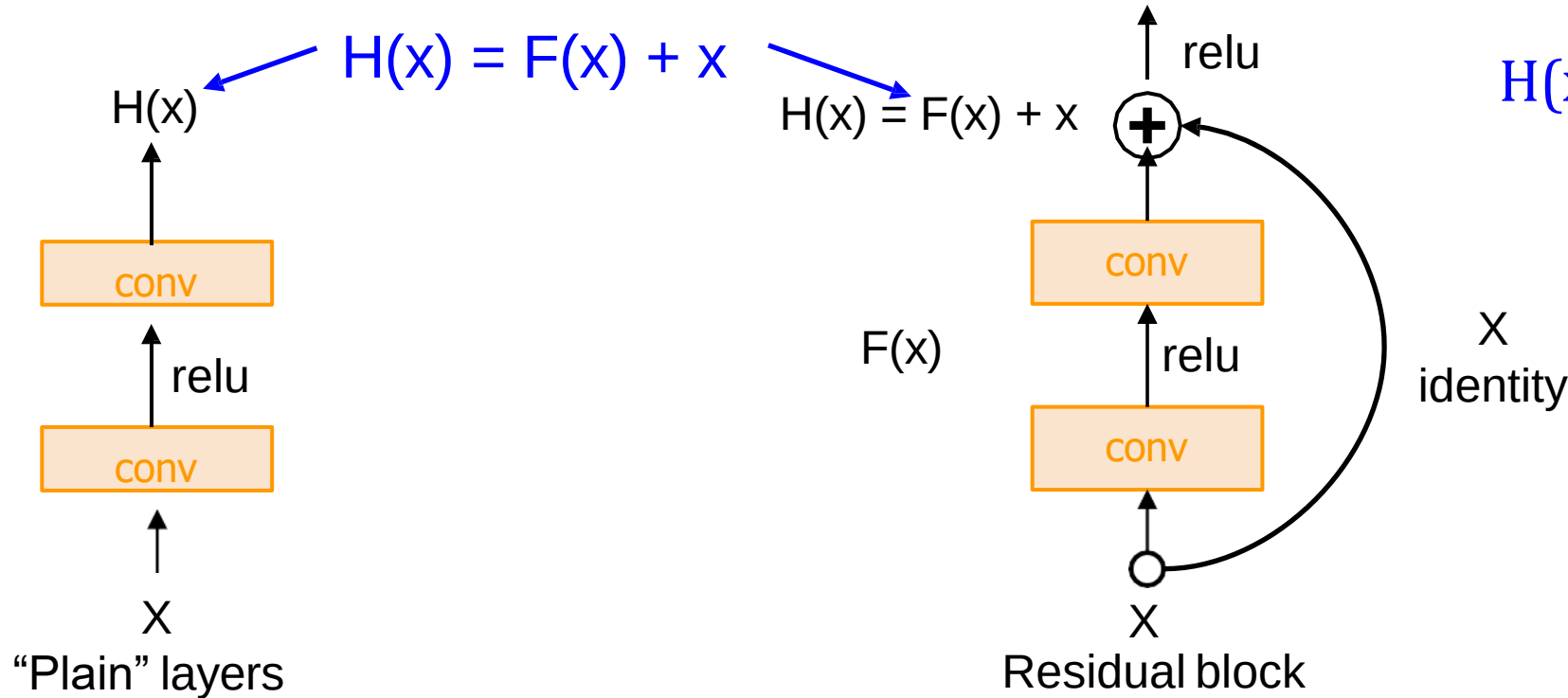


Use network layers to fit a residual mapping instead of directly trying to fit a desired underlying mapping

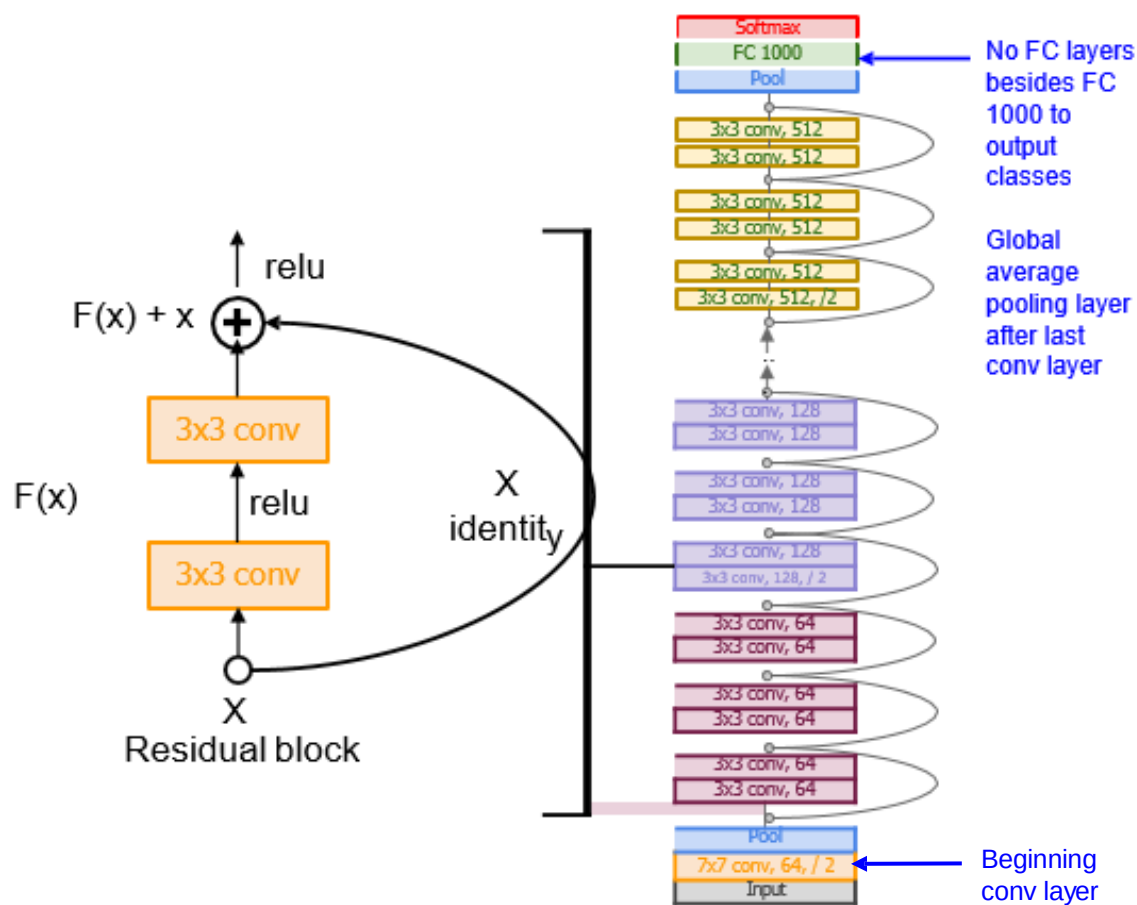
Identity mapping:

$$H(x) = x \text{ if } F(x) = 0$$

Use layers to fit **residual**
 $F(x) = H(x) - x$
instead of
 $H(x)$ directly



ResNet



- ILSVRC'15 classification winner (3.57% top 5 error)
- Swept all classification and detection competitions in ILSVRC'15 and COCO'15.
- Enable very deep models (from 20 layers to 150 and even 1000 layers!)
- No linear layers except classifier.
- Solve the problems of vanishing gradients using Residual connections
- Stack residual blocks
- Every residual block has two 3×3 conv layers
- Periodically, double # of filters and downsample spatially using stride 2. Reduce the activation volume by half
- Additional conv layer at the beginning (stem)
- No FC layers at the end (only FC 1000 to output classes)

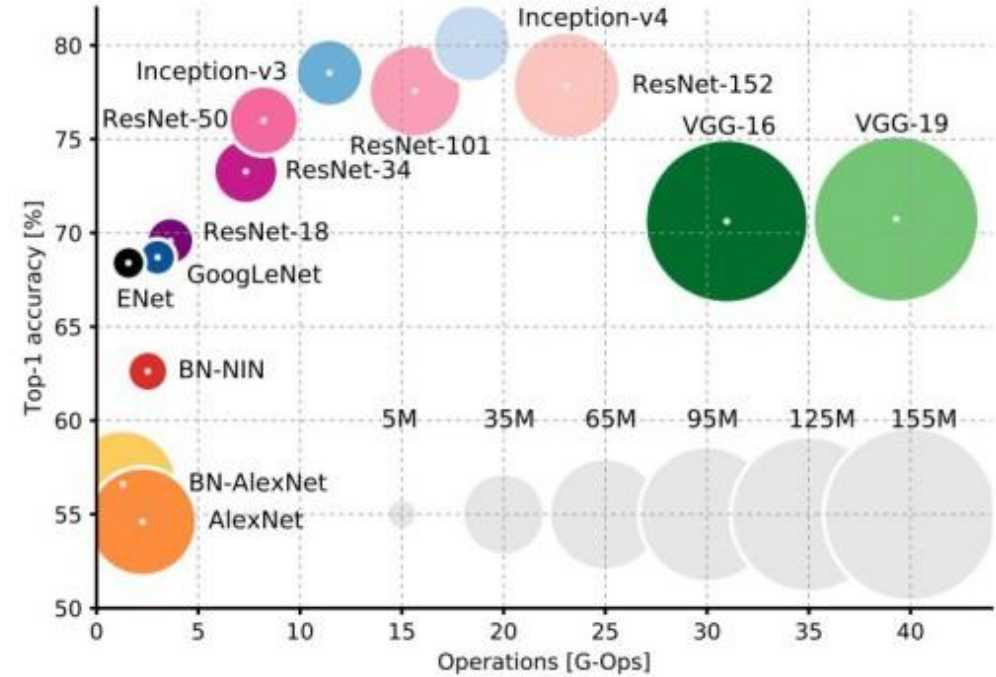
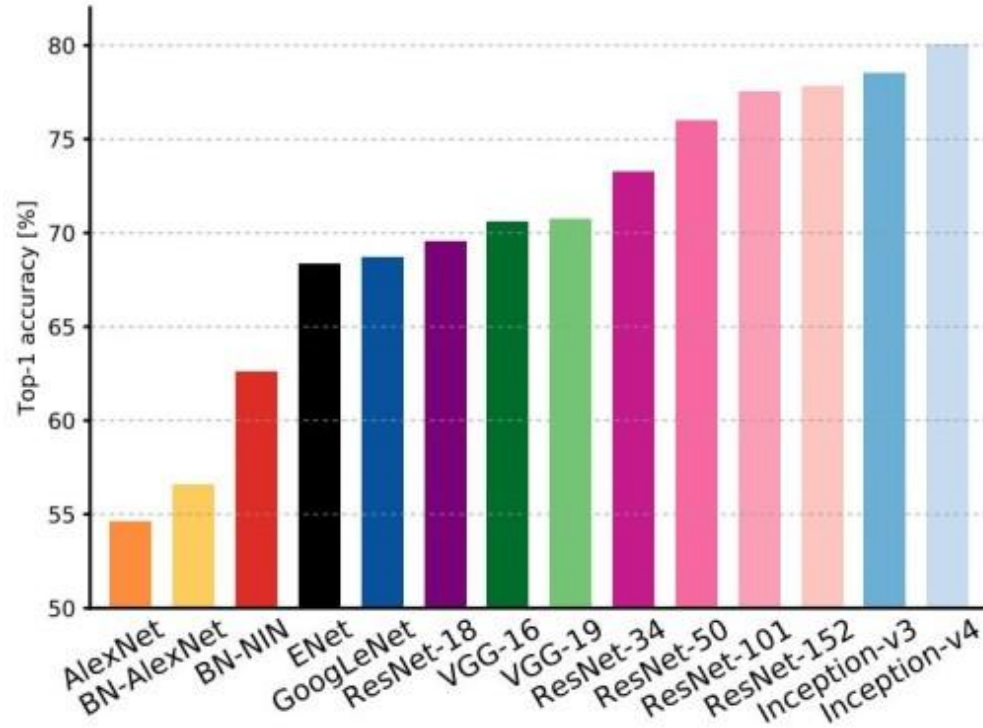
ResNet



layer name	output size	18-layer	34-layer	50-layer	101-layer	152-layer
conv1	112×112	7×7, 64, stride 2				
conv2_x	56×56	3×3 max pool, stride 2				
		$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$
conv3_x	28×28	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 8$
conv4_x	14×14	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 23$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 36$
conv5_x	7×7	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$
	1×1	average pool, 1000-d fc, softmax				
FLOPs		1.8×10^9	3.6×10^9	3.8×10^9	7.6×10^9	11.3×10^9

- **Training ResNet in practice:**
 - Batch Normalization after every CONV layer
 - Xavier initialization
 - SGD + Momentum (0.9)
 - Learning rate: 0.1, divided by 10 when validation error plateaus
 - Mini-batch size 256
 - Weight decay of 1e-5
 - No dropout used

Comparing complexity



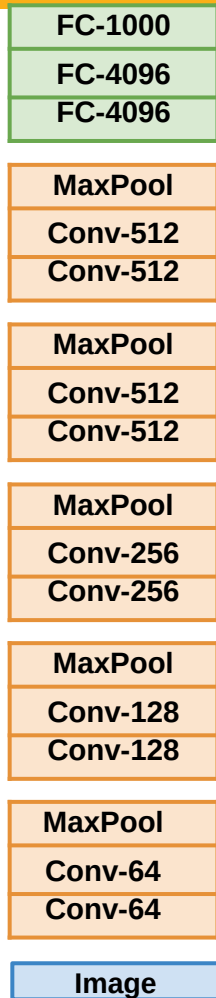
Comparing complexity



- Inception-v4: Resnet + Inception – more accuracy
- VGG: most parameters, most operations
- GoogLeNet: most efficient
- AlexNet: Smaller compute, still memory heavy, lower accuracy
- ResNet: Moderate efficiency depending on model, highest accuracy

Transfer Learning with CNNs

1. Train on Imagenet

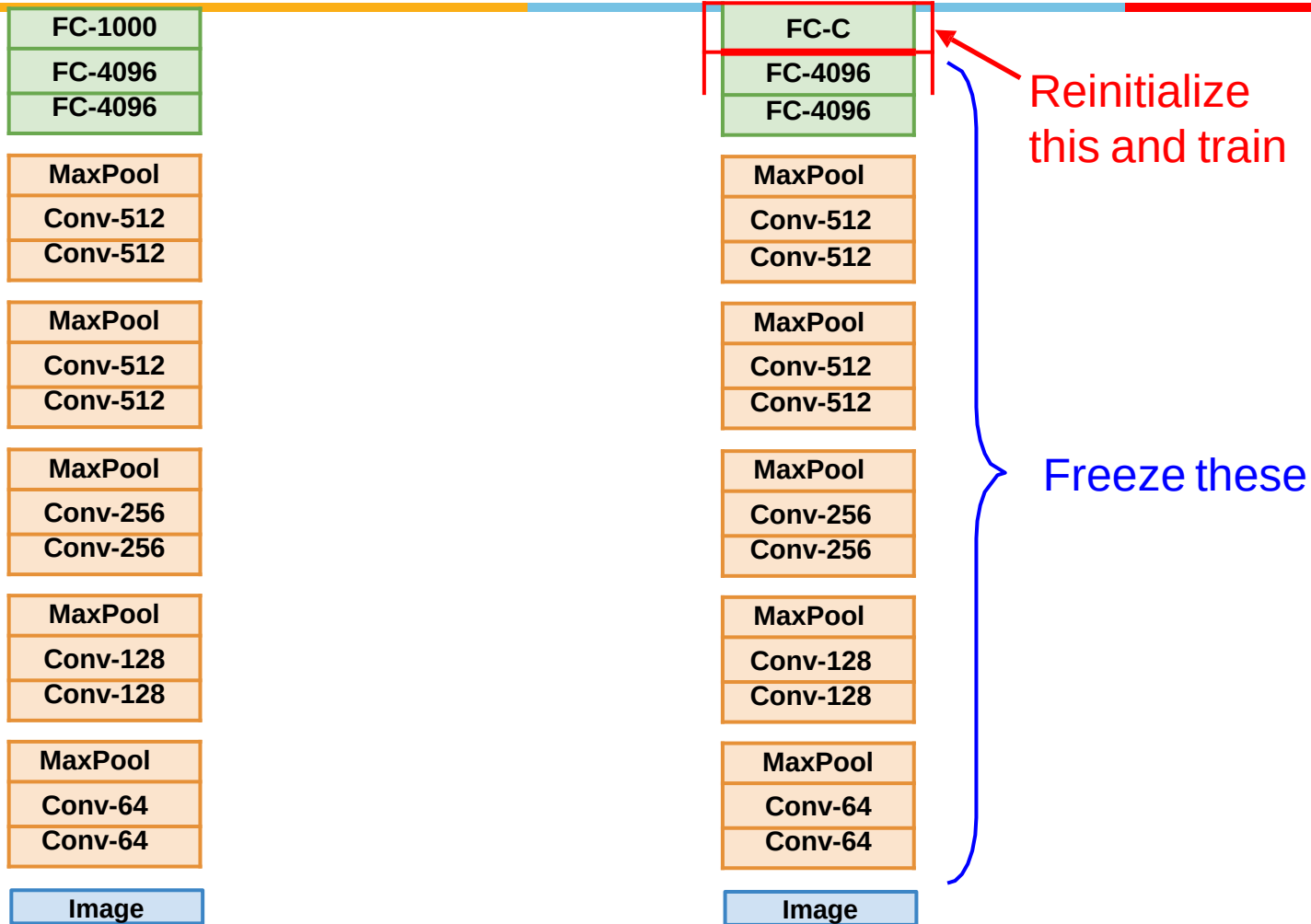


Donahue et al, "DeCAF: A Deep Convolutional Activation Feature for Generic Visual Recognition", ICML 2014
Razavian et al, "CNN Features Off-the-Shelf: An Astounding Baseline for Recognition", CVPR Workshops 2014

Transfer Learning with CNNs

1. Train on Imagenet

2. Small Dataset (C classes)

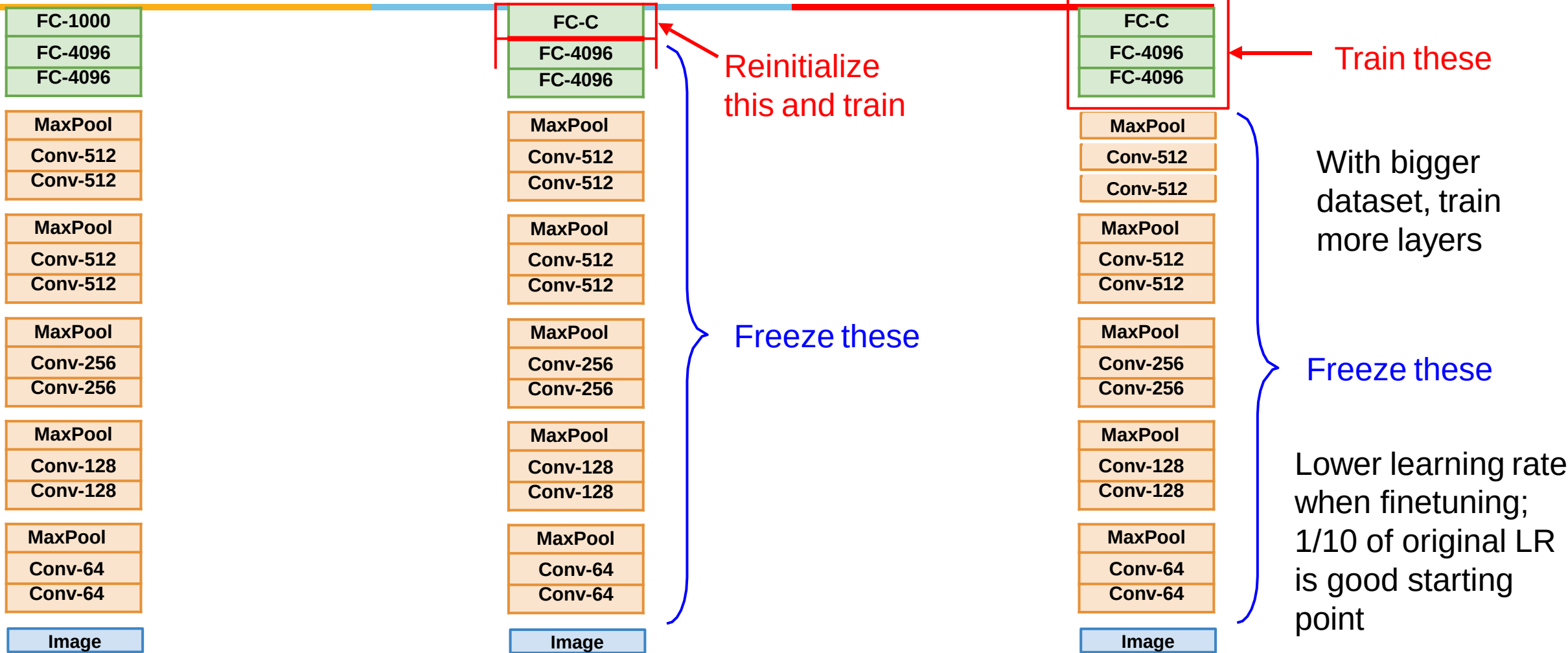


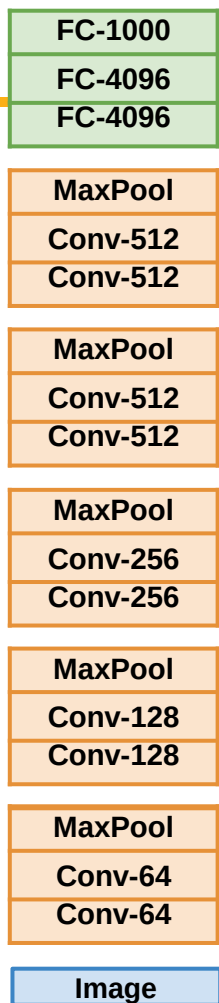
Transfer Learning with CNNs

1. Train on Imagenet

2. Small Dataset (C classes)

3. Bigger dataset





	very similar dataset	very different dataset
very little data	Use Linear Classifier on top layer	You're in trouble... Try linear classifier from different stages
quite a lot of data	Finetune a few layers	Finetune a larger number of layers or start from scratch!

References



- Hands-On Machine Learning book by Aurélien Géron
- Refer Chapter 8 of Dive into Deep Learning book